

Chapter 2. Scripts and Modules

We used the `node` command briefly in [Chapter 1](#) to explore Node's REPL mode and execute simple scripts. In this chapter, we'll learn how Node loads and executes scripts and modules. We'll start by exploring more of the options, arguments, and environment variables that can be used with the `node` command, and learning more about what we can do in a REPL session. Then we'll learn about the steps Node takes to load and execute a module.

Node CLI

The `node` command has many options that can be used to customize its behavior. It also supports arguments and environment variables to further customize what it does, and to pass data from the OS environment to Node's process environment.

Let's take a look. In the terminal, type the following:

```
$ node -h | less
```

This will output the help documentation for the command (one page at a time because we piped the output on the `less` command). I find it useful to always get familiar with the help pages for the commands I use often:

```
Usage: node [options] [ script.js ] [arguments]
       node inspect [options] [ script.js | host:port ] [arguments]

Options:
  -                    script read from stdin (default if no
                        file name is provided, interactive mode
```

```

        if a tty)
--          indicate the end of node options
--abort-on-uncaught-exception
          aborting instead of exiting causes a
          core file to be generated for analysis
--build-snapshot
          Generate a snapshot blob when the
          process exits. Currently only supported
          in the node_mksnapshot binary.
-c, --check
          syntax check script without executing
--completion-bash
          print source-able bash completion
          script
-C, --conditions=...
          additional user conditions for
          conditional exports and imports
--cpu-prof
          Start the V8 CPU profiler on start up,
:

```

The first two lines specify how to use the `node` command. Anything in square brackets is optional, which means, according to the first line, we can use the `node` command on its own without any options, scripts, or arguments. That's what we did to start a REPL session. To execute a script, we used the `node script.js` syntax (`script` can be any name there).

What's new here is that there are options and arguments that we can use with the command. Let's talk about these.

TIP

The second usage line is to start a terminal debugging session for Node. While that's sometimes useful, in [Chapter 4](#) I'll show you a much better way to debug code in Node.

In the help page, right after the usage lines, there is a list of all the options that you can use with the command. Most of these options are advanced, but knowing of their existence is a helpful reference. You should scan through this list just to get a quick idea of all the types of things that you can do with the command. Let me highlight a few of the options that I think you should be aware of.

Options and Arguments

The `--check` option (or `-c`) lets you check the syntax of a Node script without running that script. An example use of this option is to automate a syntax check before sharing code with others.

The `--eval` and `--print` options (or `-e` and `-p`) can both be used for executing code directly from the command line. I like the `-p` option more because it executes and prints (just like in the REPL mode). To use these options, you pass them a string of Node code. Here is an example:

```
$ node -p "Math.random()"
```

This is handy, as you can use it to create your own powerful commands (and alias them if you want). For example, say you need a command to generate a unique random string (to be used as a password maybe). You can leverage Node's `crypto` module in a short `-p` one-liner:

```
$ node -p "crypto.randomBytes(16).toString('hex')"
```

Pretty cool, isn't it?

NOTE

Note how the `crypto` module is available to the `-p` option without needing to require it (just like in the REPL mode).

How about a command to count the words in any file? This one will help us understand how to use arguments with the `node` command:

```
$ node -p "fs.readFileSync(process.argv[1]).toString().split(/\s+/).length" ~/.bashrc
```

Don't panic. There's a lot going on with this one. It leverages the powers of both Node and JavaScript. Go ahead and try it first. You can replace

`~/.bashrc` with a path to any file on your system.

Let's decipher this one a bit. The `readFileSync` function is part of the built-in `node:fs` module. It takes a file path as an argument and synchronously returns a binary representation of that file's data. That's why I chained a `.toString` call to it, to get the file's actual content (in UTF-8). Furthermore, instead of hardcoding the file path in the command, I put the path as the first argument to the `node` command itself and used `process.argv[1]` to read the value of that argument (see the explanation of that in the next sidebar). This enables us to use the word-counting one-liner with any file. We can alias it (without the path argument) and then use the alias with a path argument, as shown in [Figure 2-1](#):

```
$ alias count-words="node -p 'fs.readFileSync(process.argv[1]).toString().split(/\s+/).length'"
```

Then once I have the content of the file, I use JavaScript's `split` method (which is available on any string) to split the content using the `/\s+/` regular expression (which means one or more spaces). This produces an array of words, and we can then count the array items with a `.length` call to find the number of words.



```
TERMINAL
$ count-words
bash: count-words: command not found
$
$ alias count-words="node -p 'fs.readFileSync(process.argv[1]).toString().split(/\s+/).length'"
>
$
$ count-words ~/.bashrc
36
$
$
```

Figure 2-1. Aliasing a Node print one-liner

We know from the usage syntax that the `node` command can take arguments. These arguments can be any list of strings, and when you specify them, you make them available to the Node process.

The word-counting one-liner used `process.argv[1]`. The `process` object is a global scope object, and it simply represents Node's interface to the actual OS process that executes the `node` command. The `argv` property is an array that holds all the arguments you pass to the `node` command (regardless of how you're using the command). To understand that, run the following command:

```
$ node -p "process.argv" hello world
```

This will output the entire array of arguments. Node uses the first element in that array for the path of the `node` command itself, then the arguments are listed in order. That's why in the word-counting one-liner, I used the second element of `argv`.

Note that if you're executing a script, the path for that script will be the second element of `process.argv`, and the arguments (if any) will be listed starting with the third element.

The `--require` option (or `-r`) allows you to require a module before executing the main script. This is useful if you need to load a specific module before running your code or if you want to set up certain configurations or load some variable values. This one works only with CommonJS modules. For ES modules, you can use the `--import` option.

For example, let's say you have a Node project that requires the use of a module called `dotenv`, which loads environment variables from a file. Normally, you would need to include something like `require('dotenv').config()` at the beginning of your main file to use the `dotenv` module. However, with the `-r` option, you can load the module automatically without having to add it to any file:

```
$ node -r dotenv/config index.js
```

NOTE

Node supports loading environment variables from a file directly with the `--env-file` option. We'll see an example of that shortly.

The `--watch` option allows you to watch a file (and its dependencies) for changes. It automatically restarts Node when a change is detected. This is very useful in development environments. You can test it with any of the files we wrote so far. For example, to run the basic web server example from [Chapter 1](#) in watch mode, you can use the following command:

```
$ node --watch index.js
```

This will start the server in watch mode. Make a change to the *server.js* file (change the “Hello World” string, for example), and notice how the `node` command will automatically restart.

The `--test` option makes Node look for and execute code that's written for testing. Node uses simple naming conventions for that. For example, it'll look for any files named with a *.test.js* suffix, or files whose names begin with *test-*.

There are a lot more options, but most of them are for advanced use. It's good to be aware of them so that in the future you can look up if there's one particular option that might make a task you're doing simpler.

Since Node is a wrapper around V8, and V8 itself has CLI options, the `node` command accepts many V8 options as well. The list of all the V8 options you can use with the `node` command can be printed with the following:

```
$ node --v8-options | less
```

You can set JavaScript harmony flags (to turn on/off experimental features); you can also set tracing flags, customize the engine memory management, and enable or disable many other features. As with the `node` command options, it's good to know that all these options exist.

Environment Variables

Toward the end of the `node -h` output, you can see a list of environment variables like `NODE_DEBUG`, `NODE_PATH`, and many more. Environment variables are another way to customize the behavior of Node or make custom data available to the Node process (similar to command arguments).

Every time you run the `node` command, you start an OS process. In Linux, the command `ps` can be used to list all running processes. If you run the `ps` command while a Node process is running (like the basic web server example), one of the listed processes will be Node (and you can see its process ID and stop it from the terminal if you need to). Here's a command to output all process details and filter the output for processes that have the word *node* in them:

```
$ ps -ef | grep "node"
```

The `process` object represents a bridge between the Node environment and the OS environment. We can use it to exchange information between Node and the OS. In fact, when you `console.log` a message, under the hood, the code is basically using the `process` object to write a string to the OS standard output (stdout) data stream.

Environment variables are one way to pass information from the OS environment (used to execute the `node` command) to the Node environment, and we can read their values using the `env` property of the `process` object.

Here's an example to demonstrate that:

```
$ NAME="Reader" node -p "'Hello ' + process.env.NAME"
```

This will output `Hello Reader`, as shown in [Figure 2-2](#). It sets an environment variable, `NAME`, then reads its value with `process.env.NAME`. You can set multiple environment variables if you need to, either directly from the command line like this example, or using the Linux `export` command prior to executing the `node` command:

```
$ export GREETING="Hello"; export NAME="Reader"; \  
node -p "process.env.GREETING + ' ' + process.env.NAME"
```

TIP

In Linux (and macOS), you can use a semicolon to execute multiple commands on the same line and `\` to split a command into multiple lines.



```
TERMINAL bash + v [ ] [ ] ...  
● $ NAME="Reader" node -p "'Hello ' + process.env.NAME"  
Hello Reader  
○ $  
● $ export GREETING="Hello"; export NAME="Reader"; \  
> node -p "process.env.GREETING + ' ' + process.env.NAME"  
Hello Reader  
○ $  
○ $ |
```

Figure 2-2. Using environment variables in Node

You can use environment variables to make your code customizable on different machines or environments. For example, the basic web server example in [Chapter 1](#) hardcoded the port to be 3000. However, on a different machine 3000 might not be available, or you might need to run the server on a different port in a production environment. To do that, you can modify the code to use `process.env.PORT ?? 3000` instead of just `3000` (in the `listen` method) and then run the `node` command with a custom port when you need to:


```
$ PORT=4000 node index.js
```

Note that if you don't specify a port, the default port would be 3000 because I used the `??` (nullish) operator to specify a value when `process.env.PORT` does not have one. This is a common practice.

NOTE

You can't use Node's `process.env` object to change an OS environment variable. It's basically a copy of all the environment variables available to the process.

The list of environment variables shown toward the end of the `node -h` output are Node's built-in environment variables. These are variables that Node will look for and use if they have values. Here are a few examples:

`NODE_PATH`

This can be used to simplify import statements by using absolute paths instead of relative ones.

`NODE_OPTIONS`

This is an alternative way to specify the options Node supports instead of passing them to the command line each time.

`NODE_DEBUG`

This can be used to tell Node to output more debugging information when it uses certain libraries (which we specify as a comma-separated list). For example, with `NODE_DEBUG=fs,http`, Node will start outputting debugging messages when the code uses either the `node:fs` or `node:http` modules. Many packages support this environment variable. [Figure 2-3](#) shows an example of using it with the `node:http` module.

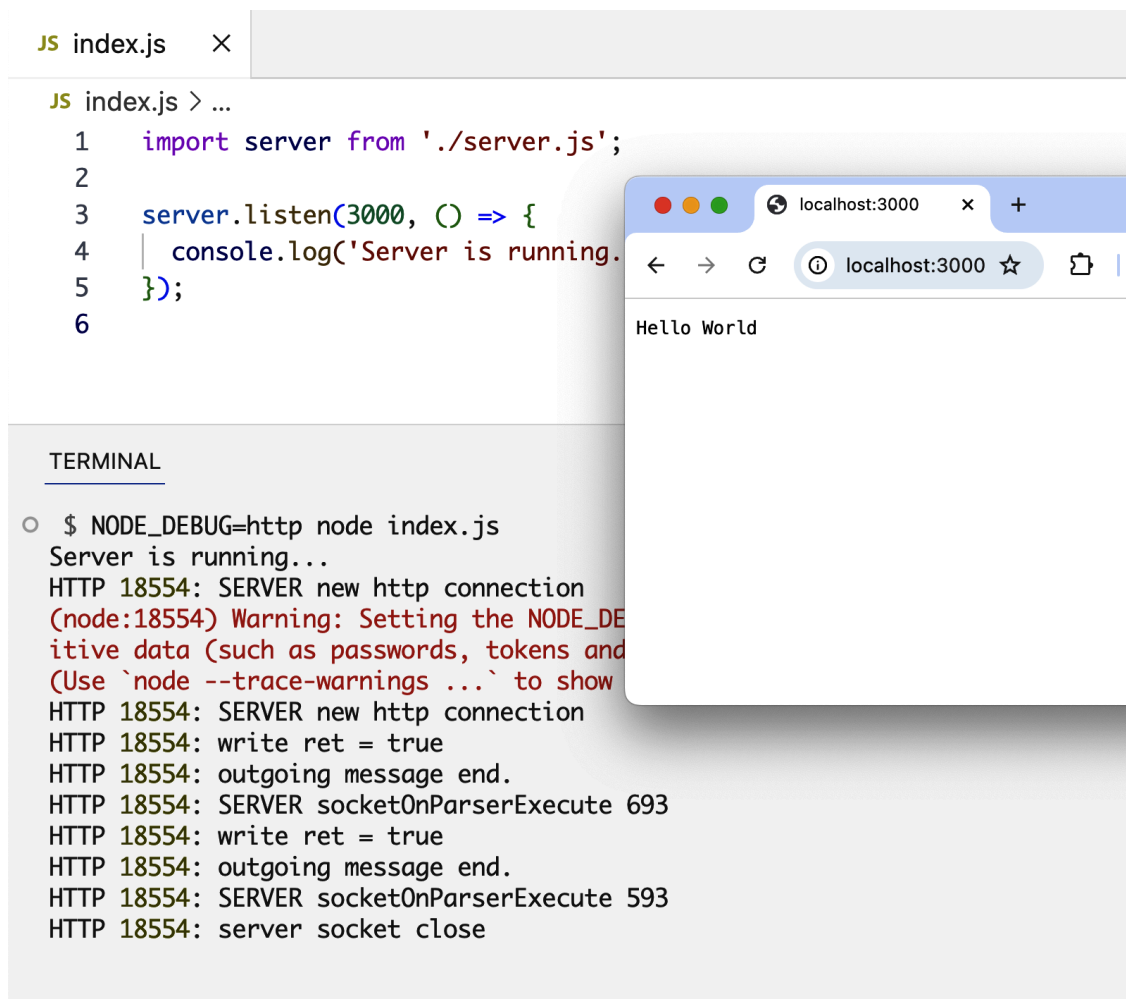


Figure 2-3. Using `NODE_DEBUG` with the `http` module

You can also put all the environment variables you need to set in a file (like an `.env` file for example) and then instruct Node to include all of the values defined in that file in the `process.env` object, using the `--env-file` option of the `node` command. Say, for example, you have the following `.env` file:

```
PORT=3000
NODE_DEBUG=fs,http
```

You can execute a Node script with these environment variables set using the following command:

```
$ node --env-file=.env script.js
```

You can use multiple environment files if you need to.

REPL Mode

In Node's REPL mode, as we learned in [Chapter 1](#), you can type any JavaScript code, and Node will execute it and automatically print its result. This is a convenient way to quickly test short JavaScript expressions (and it works for bigger code too). There are a few other helpful things you can do in REPL mode beyond the quick tests.

In REPL mode, you usually type an expression (for example, `0.1 + 0.2`) and hit Enter to see its result. You can also type statements that are not expressions, such as `let v = 21;`. In this case, when you hit Enter the variable `v` will be defined, and the REPL mode will print `undefined` since that statement does not evaluate to anything. If you need to clear the screen, you can do so with Ctrl + L.

If you try to define a function, you can write the first line and hit Enter; the REPL mode will detect that your line is not complete, and it will go into a *multiline* mode so that you can complete it, as shown in [Figure 2-4](#). Try to define a small function to test that.

A screenshot of a terminal window titled "TERMINAL". The terminal shows a Node.js REPL session. The prompt is "\$ node". The user enters ">" followed by a function definition: "> function add(x, y) {" on one line, "... return x + y;" on the next, and "... }" on the third. The prompt returns "undefined". The user enters ">" followed by a function call: "> add(4, 3)". The prompt returns "7". The user enters ">" followed by a blank line, and the prompt returns "> |", indicating multiline mode is active.

```
TERMINAL node + - [ ] [ ] ...
$ node
>
> function add(x, y) {
... return x + y;
... }
undefined
>
> add(4, 3)
7
>
> |
```

Figure 2-4. Node REPL multiline mode

The REPL multiline mode is limited, but there's an integrated basic editor available within REPL sessions as well. While in a REPL session, type

`.editor` to start the basic editor mode. Then you can type as many lines of code as you need; you can define multiple functions or paste code from the clipboard. When you're done, hit Ctrl + D to have Node execute all the code you typed in the editor.

The `.editor` command is one of many REPL commands that you can see by typing the `.help` command:

```
> .help
.break      Sometimes you get stuck, this gets you out
.clear      Alias for .break
.editor      Enter editor mode
.exit        Exit the REPL
.help        Print this help message
.load        Load JS from a file into the REPL session
.save        Save all evaluated commands in this REPL session to a file

Press Ctrl+C to abort current expression, Ctrl+D to exit the REPL
```

The `.break` command lets you get out of weird cases in REPL sessions. For example, when you paste some code in Node's multiline mode and you're not sure how many curly braces you need to get to an executable state, you can completely discard your pasted code by using a `.break` command (or pressing Ctrl + C once). This saves you from killing the whole session to get yourself out of situations like these.

The `.exit` command exits the REPL session (just like Ctrl + D).

The `.save` command enables you to save all the code you typed in one REPL session into a file. The `.load` command enables you to load JavaScript code from a file and make it all available within the REPL session. Both of these commands take a filename as an argument.

One of my favorite things about Node's REPL mode is how I can inspect basically everything that's available natively in Node without needing to require it. All the built-in modules (like `fs`, `http`, etc.) are preloaded in a REPL session and you can use the Tab key to inspect their APIs.

Just like in a terminal or editor, hitting the Tab key once in a REPL session will attempt to autocomplete anything you partially type. Try typing `cr` and hitting Tab to see it get autocomplete to `crypto`. Hitting the Tab key twice can be used to see a list of all the possible things you can type from whatever partially typed text you have. For example, type `a` and hit Tab twice to see all the available global scope objects that begin with `a`, as shown in [Figure 2-5](#).

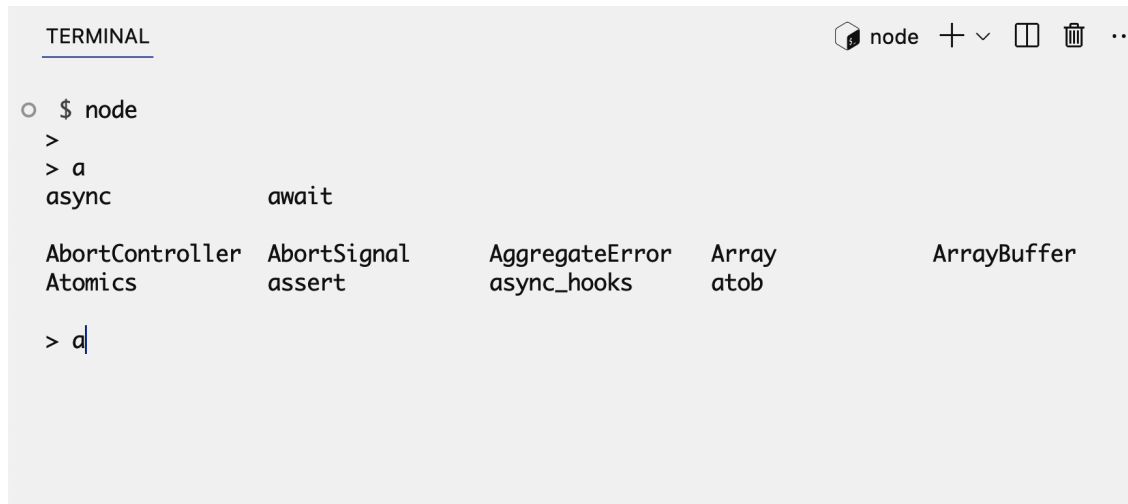


Figure 2-5. Node REPL autocomplete

This is great if you need to type less and avoid typing mistakes, but it gets better. You can use the Tab key to inspect the methods and properties available on any object. For example, type `Array.` and hit Tab twice to see all the methods and properties that you can use with the JavaScript `Array` class. This works with Node modules as well. Try it with `fs.` or `http.`.

It even works with objects that you create. For example, create an empty array using `let myArr = [];`, then type `myArr.` and hit Tab twice to see all the methods available on an array instance, as shown in [Figure 2-6](#).

```
TERMINAL node + v [] [] ...
o $ node
>
> let myArr = [];
undefined
>
> myArr.
myArr.__proto__      myArr.hasOwnProperty      myArr.isPrototypeOf
myArr.propertyIsEnumerable  myArr.valueOf

myArr.at              myArr.concat              myArr.constructor
myArr.copyWithIn     myArr.entries             myArr.every
myArr.fill           myArr.filter              myArr.find
myArr.findIndex       myArr.findLast            myArr.findLastIndex
myArr.flat           myArr.flatMap             myArr.forEach
myArr.includes        myArr.indexOf             myArr.join
myArr.keys            myArr.lastIndexOf        myArr.map
myArr.pop            myArr.push               myArr.reduce
myArr.reduceRight     myArr.reverse            myArr.shift
myArr.slice          myArr.some               myArr.sort
myArr.splice         myArr.toLocaleString      myArr.toReversed
myArr.toSorted        myArr.toSpliced           myArr.toString
myArr.unshift        myArr.values              myArr.with

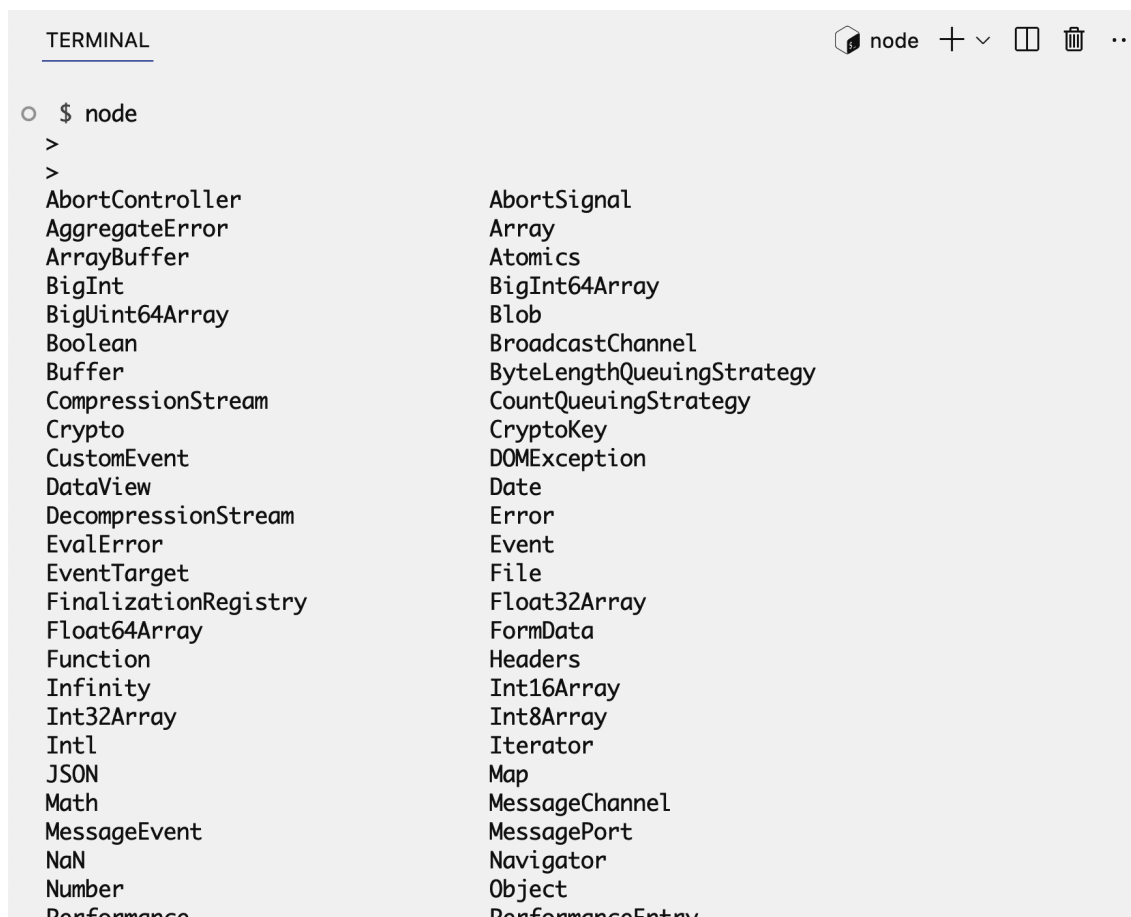
myArr.length

> myArr.|
```

Figure 2-6. Exploring methods with autocomplete

Tab discoverability works on the global level too. If you hit Tab twice on an empty line, you get a list of everything that is globally available, as shown in [Figure 2-7](#).

This is a big list, but it's a useful one. It has all the globals in the JavaScript language itself (like `Array`, `Number`, `Math`, etc.), and it has all the globals from Node (like `process`, `setTimeout`, etc.), and it also lists all the core modules that are available natively in Node (like `fs`, `http`, etc.).



```
TERMINAL node + - [ ] [X] ...
○ $ node
>
>
AbortController      AbortSignal
AggregateError       Array
ArrayBuffer          Atomics
BigInt               BigInt64Array
BigUint64Array       Blob
Boolean              BroadcastChannel
Buffer               ByteLengthQueuingStrategy
CompressionStream    CountQueuingStrategy
Crypto               CryptoKey
CustomEvent           DOMException
DataView             Date
DecompressionStream Error
EvalError            Event
EventTarget          File
FinalizationRegistry Float32Array
Float64Array         FormData
Function             Headers
Infinity             Int16Array
Int32Array           Int8Array
Intl                 Iterator
JSON                 Map
Math                 MessageChannel
MessageEvent          MessagePort
NaN                  Navigator
Number               Object
Performance           PerformanceEntry
```

Figure 2-7. Hitting Tab twice on an empty line

TIP

In the list of all global things, you'll notice an underscore character (`_`). This is a handy REPL session variable that stores the value of the last evaluated expression. For example, after executing a `Math.random()` line, you can type `_` to access that same random value. You can even use it in any place where you use a JavaScript expression. Try `let random = _;`

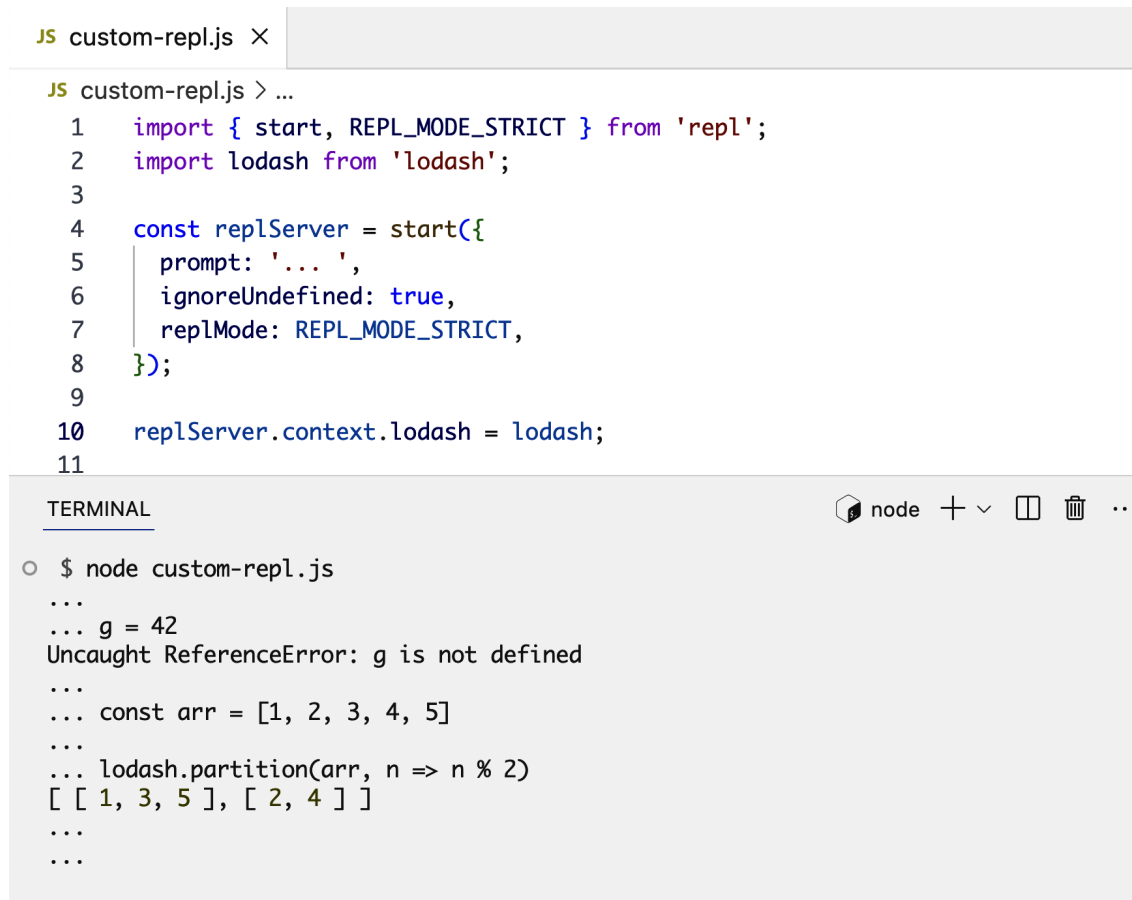
You can use the `node:repl` module to create your own custom REPL server. You can customize many things, like the prompt, the input and output streams, whether to use colors or not, and a few more options. You can also attach your own global context objects to it.

Here's a custom REPL example that'll start a REPL session with a different prompt, in strict mode, and it'll not output the return value if it's `undefined`. It'll also make the `lodash` library available globally in your custom REPL sessions (see [Figure 2-8](#)):

```
import { start, REPL_MODE_STRICT } from 'repl';
import lodash from 'lodash';

const replServer = start({
  prompt: '... ',
  ignoreUndefined: true,
  replMode: REPL_MODE_STRICT,
});

replServer.context.lodash = lodash;
```



The screenshot shows a code editor with a file named `custom-repl.js`. The code defines a custom REPL server using the `repl` module and `lodash`. The server is configured with a prompt of `'... '`, `ignoreUndefined: true`, and `REPL_MODE_STRICT`. The `lodash` module is added to the server's context. Below the code editor, a terminal window shows the command `$ node custom-repl.js` being executed. The terminal output shows the prompt `... g = 42`, followed by an error message `Uncaught ReferenceError: g is not defined`. The prompt then changes to `... const arr = [1, 2, 3, 4, 5]`, and the next prompt is `... lodash.partition(arr, n => n % 2)`. The output of the `partition` function is displayed as `[ [ 1, 3, 5 ], [ 2, 4 ] ]`.

```
JS custom-repl.js > ...
1 import { start, REPL_MODE_STRICT } from 'repl';
2 import lodash from 'lodash';
3
4 const replServer = start({
5   prompt: '... ',
6   ignoreUndefined: true,
7   replMode: REPL_MODE_STRICT,
8 });
9
10 replServer.context.lodash = lodash;
11
```

TERMINAL

```
$ node custom-repl.js
...
... g = 42
Uncaught ReferenceError: g is not defined
...
... const arr = [1, 2, 3, 4, 5]
...
... lodash.partition(arr, n => n % 2)
[ [ 1, 3, 5 ], [ 2, 4 ] ]
...
...
```

Figure 2-8. Using a custom REPL server

Node Modules

The word *module* means a reusable piece of code. Something you can include and use in any application, as many times as you need.

In Node, the word *script* is usually used for a piece of code that's executed once with the `node` command. Any other files or folders that are re-

quired or imported are what's referred to as modules.

When you specify a module as a dependency, Node goes through a few key steps to complete the module loading process: resolution and reading of the module contents, isolating the module scope, executing the module code, and caching the module.

Resolving Modules

Node uses the following procedure to determine how to find a module that is being imported.

If the module name does not start with a `.` (denoting a relative path) or a `/` (denoting an absolute path), Node will first check if the module is a built-in one. If it is, it'll load and execute it directly.

If the module is not built-in, Node will look for it under *node_modules* folders starting from the location where the importing module is and going up in the folders hierarchy. For example, if the importing module is in */User/samer/efficient-node/src*, Node will first look under *src* for a *node_modules* folder. If it does not find one, it'll look next under *efficient-node*, and so on, all the way to the root path.

You can use this lookup procedure to localize module dependencies by having multiple *node_modules* folders in your project, but that generally increases the complexity of the project. You can also use this lookup procedure to have multiple projects share a *node_modules* folder by placing that folder in a parent folder common to all projects, or even have a global *node_modules* folder for all projects on one server. While this might be useful in some cases, having a single *node_modules* folder per project is the standard and recommended practice.

If the imported module starts with a `.` or `/`, Node will look for it in the folder or file specified by the relative or absolute path.

TIP

For CommonJS modules, if you set the `NODE_PATH` environment variable before executing a script, Node will first look for any required modules in the paths specified by `NODE_PATH` (which can be a single path or multiple paths separated by a comma). This can be useful to use short absolute paths instead of confusing relative ones.

If you need to only resolve the module and not execute it, you can use the `require.resolve()` function for CommonJS modules, or the `import.meta.resolve()` function for ES modules. These functions do not load the module. They just verify that it exists, and they'll throw an error if it does not.

Loading Modules

Once the path of a module is resolved successfully, Node will read the content of the module and determine its type.

A module can be a CommonJS module or an ES module. Supported file extensions are `.js`, `.cjs`, and `.mjs`. It can be a single file or a directory with a `package.json` that specifies what files in the directory can be imported.

A module can also be a JSON file (`.json` extension). When you import a JSON file, you get a JavaScript object representing the data in that JSON file:

```
// In CommonJS modules:
const data = require('./file.json');

// In ES modules with static import:
import data from './file.json'
  with { type : 'json' };

// In ES modules with dynamic import:
const { default: data } = await import('./file.json', {
  with: { type: 'json' },
});
```

TIP

The `with` keyword in this example is used to specify the `type` import attribute. The *import attributes* feature gives the runtime instructions on how a module should be loaded. It's a security standard to prevent executing malicious code. It can be used with other module types as well (for example, a “css” module in a browser).

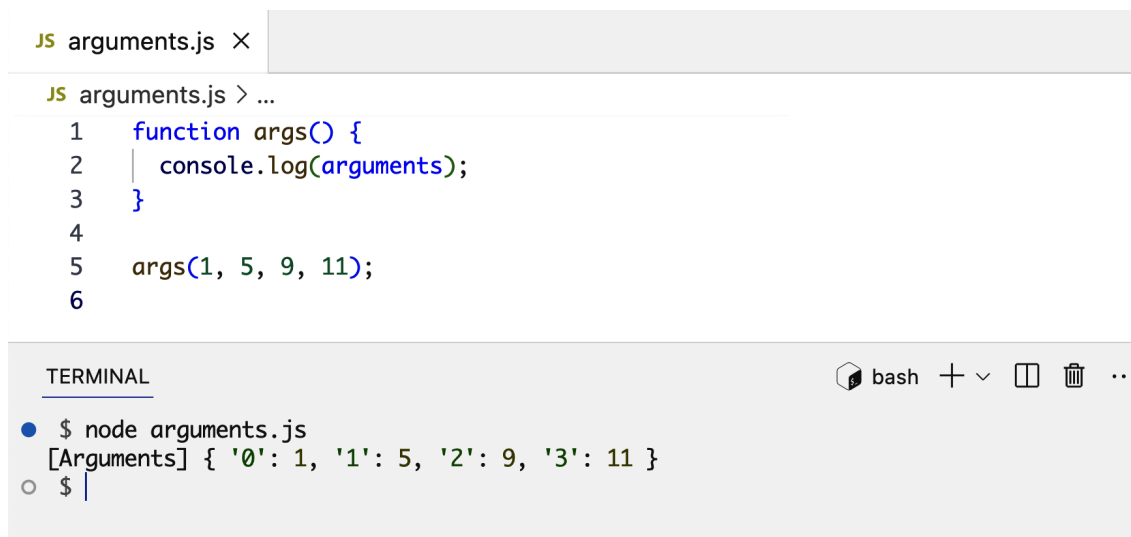
A module can also be a Node add-on compiled file. Node add-ons are dynamically linked objects implemented in a low-level language like C or C++ and compiled to be loaded as ordinary Node modules. Node has an API known as *Node-API* that's dedicated to building native add-ons. It's independent from the underlying JavaScript runtime. If you need a module with high performance, or you need it to access system resources or integrate with C or C++ libraries, you can use Node-API to build an add-on and use it as you would use any other built-in Node module.

WARNING

Add-ons are not supported with ES module imports. They can instead be loaded using the `module.createRequire()` function.

Scoping Modules

JavaScript functions can be called with any number of arguments. The `arguments` keyword can be used to access the list of all arguments a function is called with (see [Figure 2-9](#)).



The image shows a code editor window with a file named `arguments.js`. The code defines a function `args()` that logs the `arguments` object to the console. It is then called with four arguments: `1, 5, 9, 11`. Below the code editor is a terminal window. The terminal shows the command `$ node arguments.js` being executed, followed by the output: `[Arguments] { '0': 1, '1': 5, '2': 9, '3': 11 }`. The terminal also shows a new prompt `$` for the next command.

```
JS arguments.js X
JS arguments.js > ...
1 function args() {
2   | console.log(arguments);
3   }
4
5   args(1, 5, 9, 11);
6

TERMINAL bash + v [ ] [ ] ...
● $ node arguments.js
  [Arguments] { '0': 1, '1': 5, '2': 9, '3': 11 }
○ $
```

Figure 2-9. The implicit `arguments` object

TIP

If you need to have a function with a dynamic number of arguments, you should use explicit *rest parameters* instead of the implicit `arguments` keyword.

Node wraps all CommonJS modules with a function to give them a private scope. That wrapping function is called with five implicit arguments. To see that in action, print the value of the `arguments` keyword in the top-level scope of a CommonJS module, as shown in [Figure 2-10](#).

```
JS module.cjs ×
JS module.cjs > id
1 exports.id = '123';
2
3 console.dir(arguments, { depth: 1 });
4

TERMINAL
$ node module.cjs
[Arguments] {
  '0': { id: '123' },
  '1': [Function: require] {
    resolve: [Function],
    main: [Object],
    extensions: [Object: null prototype],
    cache: [Object: null prototype]
  },
  '2': {
    id: '.',
    path: '/Users/samer/efficient-node',
    exports: [Object],
    filename: '/Users/samer/efficient-node/module.cjs',
    loaded: false,
    children: [],
    paths: [Array],
    [Symbol(kIsMainSymbol)]: true,
    [Symbol(kIsCachedByESMLoader)]: false,
    [Symbol(kIsExecuting)]: true
  },
  '3': '/Users/samer/efficient-node/module.cjs',
  '4': '/Users/samer/efficient-node'
}
```

Figure 2-10. CommonJS module wrapping

These five implicit arguments are (in order): `exports`, `require`, `module`, `__filename`, and `__dirname`. When you use these within a CommonJS module, you are not using a global variable; you're using an argument from the implicit wrapping function.

The `exports`, `require`, and `module` arguments are Node's way to manage a CommonJS module's API and its dependencies. The `__filename` value has the full path of the module file. The `__dirname` value has the path to the directory where the module file is located.

Similar to CommonJS module wrapping, ES modules are executed in an implicit scope, but there is no wrapping function, and the five implicit arguments are not defined at all. Instead, an ES module API and dependencies are managed with `import` / `export` statements.

If you need to access the filename or directory name of an ES module, you can use `import.meta.filename` and `import.meta.dirname`.

With this scoping in modules, all the variables you define in a module are local to that module. If you need to define a global variable, you can use the global scope object `globalThis`. Any properties you add to that object become global variables. It's good to know that you can do that, but you should avoid using global variables as they can be problematic for many reasons.

Executing Modules

This is the step where Node will execute the code in a module and finalize its dependencies and exports.

One common coding practice is to put any configurable variables that are used to seed or run an application into their own modules. An example of such configurable variables are the `PORT` and `HOST` on which a web server will run.

Let's create a *config.cjs* file to host these two configurable variables. The *.cjs* extension makes it a CommonJS module that will be wrapped for scoping. This module will have the five implicit arguments.

NOTE

I'll provide the equivalent ES module syntax below the CommonJS module syntax. You can use the *.mjs* extension if you want to test the examples with ES modules.

As shown in [Figure 2-11](#), the `exports` argument will start out as an empty object.



Figure 2-11. The `exports` argument

To define the API of the *config.cjs* module, we just define properties on the `exports` object. Properties can be static values or any type of object in JavaScript (like a function, a class, or a promise):

```
console.log('Loading config.cjs');

exports.PORT = process.env.PORT ?? 3000;
exports.HOST = process.env.HOST ?? 'localhost';
exports.SERVER_URL = (
  protocol = process.env.PROTOCOL ?? 'http',
) => `${protocol}://${exports.host}:${exports.PORT}`;
```

```
// In ES modules
export const PORT = process.env.PORT ?? 3000;
export const HOST = process.env.HOST ?? 'localhost';
export const SERVER_URL = (
  protocol = process.env.PROTOCOL ?? 'http',
) => `${protocol}://${exports.host}:${exports.PORT}`;
```

Note how I used `process.env` variables to make the configurations customizable on different environments. I also made `SERVER_URL` a function that receives a `protocol` argument, which is customizable through the environment as well. Making a configuration value a function allows it to be customizable at runtime.

When we require this `config.cjs` module in another module, the `require` function call returns the `exports` object. Let's test that in an `index.cjs` file:

```
const config = require('./config.cjs');
console.log(config);

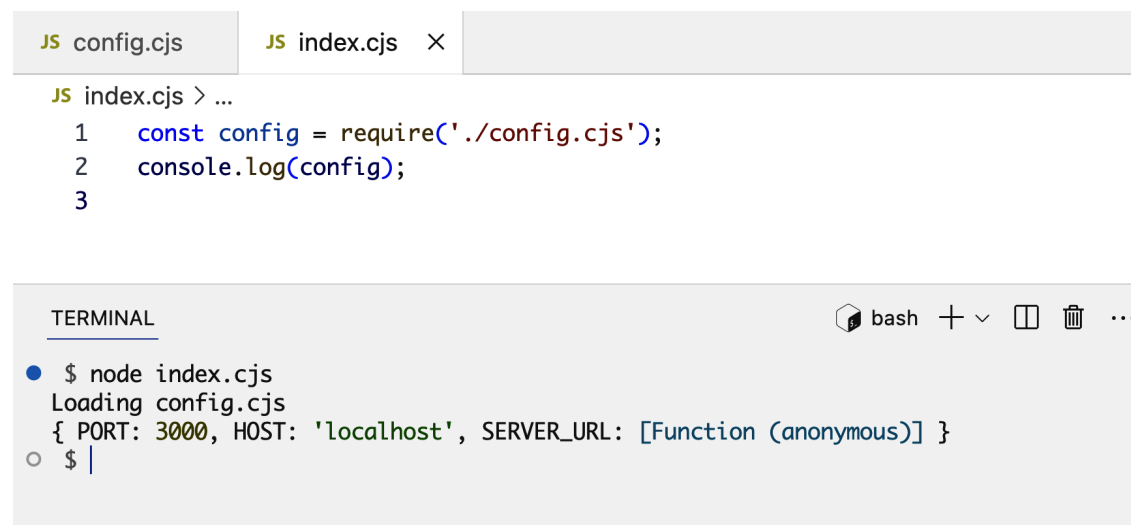
// Or we can use destructuring
// const { PORT, HOST } = require('./config.cjs');
```

```
// In ES modules
import * as config from './config.mjs';
console.log(config);

// Or we can use named imports
// import { PORT, HOST } from './config.cjs';
```

[Figure 2-12](#) shows the output.

Now we can say that the `index.cjs` module *depends* on the `config.cjs` module. This is where the term *dependency management* comes from. We are managing the dependencies of a module here and bringing one module's API to use in another module.



```
JS config.cjs JS index.cjs ×
```

```
JS index.cjs > ...
1  const config = require('./config.cjs');
2  console.log(config);
3
```

```
TERMINAL bash + ▢ 🗑️ ...
● $ node index.cjs
Loading config.cjs
{ PORT: 3000, HOST: 'localhost', SERVER_URL: [Function (anonymous)] }
○ $ |
```

Figure 2-12. Requiring a CommonJS module

The `exports` argument in CommonJS modules is actually an alias to `module.exports`. The latter is what's returned when we invoke the `require` function. In some cases, you might need the top-level API object

to be a function or a class, or anything else that's not a simple aliased object. In these cases, you'll need to change the value of `module.exports` itself to define your special API.

For example, let's say that we want all the configuration properties to be the result of executing a function. This might be helpful for testing as we can mock the configuration function differently for different tests. To make the top-level API object a function, you need to use `module.exports`. Here's an example of how we can do that for *config.cjs*:

```
module.exports = () => {
  return {
    PORT: process.env.PORT ?? 3000,
    HOST: process.env.HOST ?? 'localhost',
    SERVER_URL: (protocol = process.env.PROTOCOL ?? 'http') =>
      `${protocol}://${exports.host}:${exports.PORT}`,
  };
};
```

```
// In ES modules
export default () => {
  return {
    PORT: process.env.PORT ?? 3000,
    HOST: process.env.HOST ?? 'localhost',
    SERVER_URL: (protocol = process.env.PROTOCOL ?? 'http') =>
      `${protocol}://${exports.host}:${exports.PORT}`,
  };
};
```

With that, to use the configuration value in *index.cjs*, we'll need to invoke what we get from the `require` function:

```
const config = require('./config.cjs');

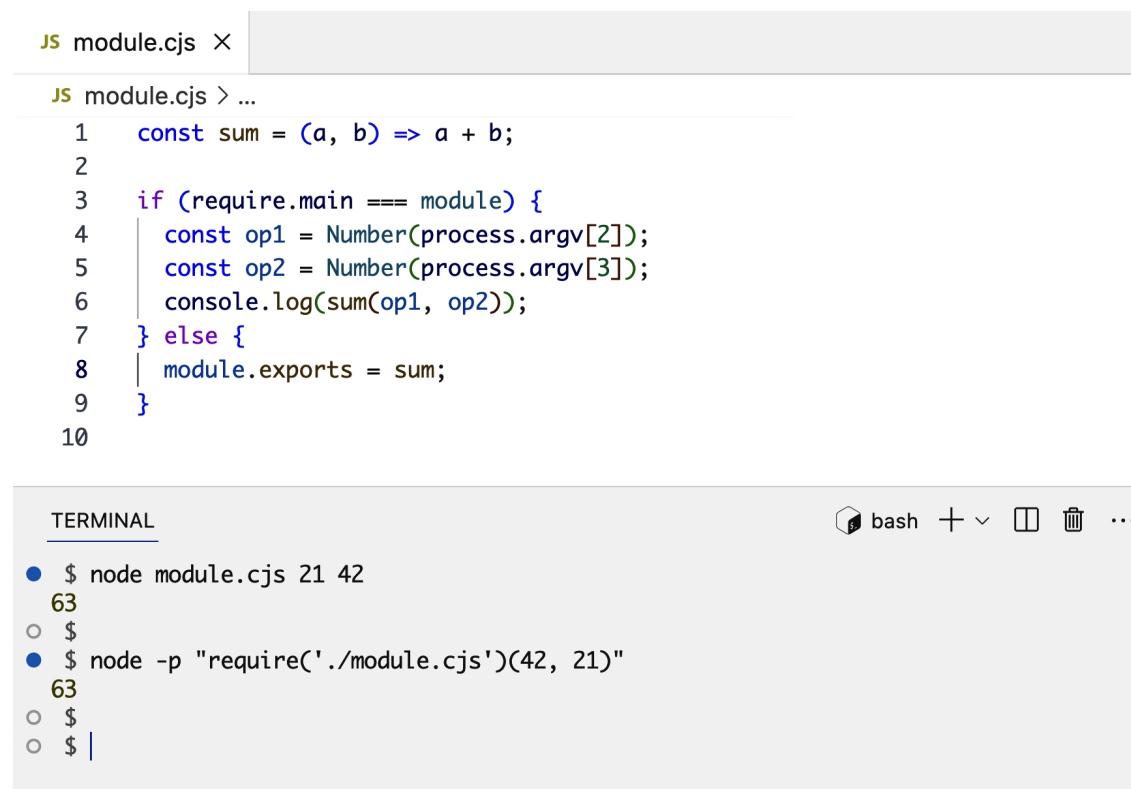
console.log(
  config(), // Note how we are invoking this
);
```

```
// In ES modules:
import config from './config.mjs';

console.log(
  config(), // Note how we are invoking this
);
```

This method is often helpful when you need to use the *dependency injection* design pattern, which is when some modules are injected into other modules to create more flexibility and make modules more reusable.

If you need to make a Node module executable from the CLI as a script, you can use the `require.main` property to check if the module is being run directly. The `require.main` value will equal the `module` argument in that case. [Figure 2-13](#) has an example of a simple module using that check to determine what to do.



The image shows a code editor window titled "JS module.cjs" with a close button. The code defines a function `sum(a, b)` and checks if the module is being run directly using `require.main === module`. If true, it processes command-line arguments and logs the result. If false, it exports the `sum` function. Below the code is a terminal window titled "TERMINAL" with a "bash" icon and window controls. It shows two commands being executed: `node module.cjs 21 42` and `node -p "require('./module.cjs')(42, 21)"`. The first command results in the output `63`, and the second command also results in `63`.

```
JS module.cjs ×
JS module.cjs > ...
1  const sum = (a, b) => a + b;
2
3  if (require.main === module) {
4    const op1 = Number(process.argv[2]);
5    const op2 = Number(process.argv[3]);
6    console.log(sum(op1, op2));
7  } else {
8    module.exports = sum;
9  }
10

TERMINAL bash + v □ 🗑️ ...
● $ node module.cjs 21 42
63
○ $
● $ node -p "require('./module.cjs')(42, 21)"
63
○ $
○ $ |
```

Figure 2-13. The `require.main` check

WARNING

ES modules have no equivalent simple check to determine if they are run directly, but `import.meta.url` can be used along with `process.argv` to do a similar check. The `es-main` npm package has a good implementation of that.

Caching Modules

To understand another concept about how Node modules work, let's repeat the `require` line in `index.cjs` multiple times:

```
require('./config.cjs');  
require('./config.cjs');  
require('./config.cjs');
```

Given these three `require` lines, when we execute `index.cjs`, how many times will the “Loading config.cjs” line in `config.cjs` be outputted?

The answer is not three times. It'll be outputted only once, as shown in [Figure 2-14](#).



Figure 2-14. Node module caching

Both CommonJS modules and ES modules in Node are cached after the first call. A module is executed the first time you import it; then when you import it again, Node loads it up from a cache.

If you look at a frontend application like React, for example, all component files import the `React` module, and that's OK, because only the first import will do the work; the rest will use the cache.

But what if you do want the `console.log` message to show up every time we require *config.cjs*?

You can make the top export of *config.cjs* a function instead of an object, put all the code there inside the function, and call that function every time you need the code to be executed. The cache, in that case, will cache the definition of the function.

Summary

Node CLI has many powerful options that we can control. We can pass arguments to it and set environment variables before running it. Both of these options allow us to pass data from the OS environment to a running Node process. Node's `process` object is the bridge.

Node's REPL mode is a good way to test simple expressions, explore everything you can use in Node, and take a quick look at the API of anything, including core modules, installed modules, and even objects you instantiate.

CommonJS modules in Node are implicitly wrapped in a function and are passed five arguments. ES modules have a private scope as well.

We use the `exports` object in CommonJS modules or `export` statements in ES modules to define the API of a module. Modules that need to depend on other modules use the `require` function or `import` statements to access a dependency API.

Node manages a cache for all modules. To discover where a module is, Node follows a predefined set of rules depending on the path of the module. A path can be a relative one, an absolute one, or just a name. For the latter case, Node looks for the module in *node_modules* folders.

In the next chapter, we'll do a deep dive into how Node handles asynchronous operations and learn about the event-driven nature of Node modules.